

Streaming Latency Benchmarks: REDIS Streams vs Apache KAFKA for Real-Time Sports Data Feeds

Journal Title
XX(X):1–12
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Gustavo Pedro Ricou¹

Abstract

We ask a question that the streaming-systems and sports-analytics literatures leave open: *does the choice of streaming infrastructure actually degrade an in-play football decision, and if so, when?* Using the publicly available StatsBomb open dataset (1. Bundesliga 2023/24, 34 matches), we replay real event feeds through Apache KAFKA and REDIS Streams and couple the *measured* event-delivery latency to an in-play win-probability model through the Age-of-Information lens, quantifying the win-probability staleness that each architecture imposes. To our knowledge this is the first infrastructure → decision-quality analysis on open sports data.

The study is also a cautionary account of measurement validity. Three distinct instrumentation defects — a process-relative cross-process clock, a saturating replay speed, and an asymmetric load generator (synchronous per-event KAFKA sends versus asynchronous REDIS dispatch) — each *reversed* the apparent winner before correction. We introduce the Time-to-Insight (TTI) metric, decomposed into producer scheduling lag and transport, and fix all three defects before drawing any conclusion.

Our central finding is that the answer turns on one condition, and not the one the literature emphasises. When broker and consumer are *co-located*, KAFKA and REDIS are statistically *equivalent* — not merely indistinguishable: two one-sided tests against a pre-specified one-broadcast-frame margin establish equivalence at every concurrency level tested (≈ 10 ms median TTI), neither system shows a concurrency effect, and a goal's win probability is stale by ≈ 0.014 probability-seconds regardless of backend. Football event data is intrinsically low-rate (0.44 events per second per match), so a single REDIS node serves the equivalent of thousands of simultaneous matches before its single-threaded serialization becomes visible. Under those conditions infrastructure choice is irrelevant to decision quality, contradicting the industry framing of a latency-decided “2-second edge”.

Introducing network latency reverses this. Injecting one-way delay identically at both brokers, KAFKA's latency tracks the delay (94 ms at 50 ms injected) whereas single-node REDIS amplifies it by two to three orders of magnitude (75 s at 50 ms injected), because REDIS Streams consumption is round-trip bound while KAFKA amortises delay across batched fetches. Equivalence survives only to ≈ 5 ms; by 20 ms the decision-staleness gap is a factor of 275 (0.058 vs. 15.97 probability-seconds), which is decision-corrupting. The practical guidance is therefore conditional: co-locate consumers with the broker and the choice does not matter, but for any consumer a network hop away KAFKA is strongly preferable. Methodologically, the episode is a caution — four measurement defects (a process-relative clock, a saturating replay speed, an asymmetric load generator, and a concurrency axis that was covertly a throughput axis) each reversed the apparent winner, and the loopback deployment hid the one difference that matters most.

Keywords

Real-time sports analytics, football, in-play win probability, Age of Information, streaming systems, Apache Kafka, Redis Streams, latency benchmarking, decision quality, reproducible research

Introduction

The rapid digitization of sports has created an insatiable demand for real-time analytics. Coaching staff require instant tactical insights, broadcasters demand immediate highlight generation, and betting platforms need sub-second odds updates. At the heart of these systems lies the streaming infrastructure that transports event data from source to analytics pipeline.

Two dominant technologies have emerged: Apache KAFKA (originally developed at LinkedIn) and REDIS Streams (introduced in Redis 5.0). KAFKA is a distributed event streaming platform designed for high-throughput, fault-tolerant data pipelines. REDIS Streams provides a

lightweight, in-memory streaming solution with persistent storage capabilities.

While numerous studies have compared these technologies, the literature lacks sports-specific evaluations. Sports

¹School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland

Corresponding author:

Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland.

Email: pedrorig@tcd.ie

data presents distinct challenges: high-frequency bursts during active play, ordered event delivery requirements, and strict latency constraints.

Contributions. This paper makes four contributions. (1) We pose and answer, on open data, the question of *when* streaming-infrastructure latency actually degrades an in-play football decision, by coupling measured event-delivery latency to a calibrated in-play win-probability model through the Age-of-Information lens—to our knowledge the first infrastructure → decision-quality analysis in sports analytics. (2) We provide a fair, artifact-free Apache KAFKA vs. REDIS Streams latency benchmark on the StatsBomb open dataset, with fully open load-generation code. (3) We document four design defects—a process-relative clock, a saturating replay speed, an asymmetric load generator, and a concurrency axis that was covertly a throughput axis—each of which reversed the apparent winner, offering a reusable protocol for honest streaming benchmarks. (4) We identify the condition that actually decides the question: with consumers co-located the two backends are statistically equivalent, but because edis Streams consumption is round-trip bound, ordinary network latency turns that equivalence into a factor-of-275 gap in decision staleness, so placement rather than brand is the design variable that matters.

Sports-Specific Latency Requirements

Real-time sports analytics imposes domain-specific latency constraints that vary by use case and stakeholder. As summarized in Table 1, different applications demand fundamentally different performance characteristics. Betting platforms require sub-500ms latency for in-play odds updates, as even a 2-second delay can erode user trust and cost millions in missed wagers [Solutions \(2025\)](#); [Ververica \(2025\)](#). Broadcasting applications can tolerate up to 3 seconds for live video synchronization [OptiView \(2025\)](#); [Promwad \(2025\)](#), while coaching decision systems need <500ms to enable real-time tactical adjustments [Pappas et al. \(2020\)](#); [Sports \(2023\)](#).

The concept of an “actionability window” formalizes this requirement: the maximum latency within which insights can still influence decisions. These budgets, summarised in Table 1, are however *asserted* by practitioners rather than derived: they say how fast a pipeline should be, not what missing the budget costs. That is precisely the gap RQ4 addresses. Our decision-staleness metric replaces the assumed threshold with a measured quantity—how much probability mass a decision-maker holds incorrectly, and for how long—so the question becomes not “did we meet 500 ms?” but “how wrong was the forecast, and for how long, because we did not?” We therefore use Table 1 as the practitioner context our results are interpreted against (Section), not as the study’s success criterion.

Research Questions

This study addresses four research questions, culminating in the decision-quality question that motivates the paper:

RQ1 (single feed): For a single live match feed, does the choice of streaming backend (Apache KAFKA vs. REDIS Streams) materially affect Time-to-Insight (TTI)?

Table 1. Sports Analytics Latency Requirements by Use Case and Stakeholder

Use Case	Stakeholder	Latency Range	Critical
Betting Platforms			
Live Odds Update	Betting Platform	100–500ms	< 5
In-Play Betting	Betting Platform	50–200ms	< 2
Broadcasting			
Live Video Sync	Broadcaster	500–3000ms	< 3
Live Stats Overlay	Broadcaster	100–1000ms	< 1
Interactive Features	Broadcaster	200–500ms	< 5
Coaching			
Tactical Adjustment	Coach	200–800ms	< 8
Player Substitution	Coach	500–1500ms	< 1
Fan Applications			
Push Notifications	Fan	1000–3000ms	< 3
Live Updates	Fan	500–2000ms	< 2
Post-Match Analysis			
Initial Analysis	Analyst	5000–10000ms	< 10

RQ2 (concurrency): How does TTI scale as one broker host serves $N \in \{1, 5, 10, 20\}$ concurrent match feeds, and do the two backends differ?

RQ3 (fairness and durability): With payload and protocol overheads controlled for fairness, what latency cost do stronger durability settings (KAFKA `acks=all`; REDIS `appendfsync always`) impose?

RQ4 (decision degradation): When measured delivery latency is translated into an in-play win-probability estimate through the Age-of-Information lens, does it degrade an in-play football decision, and in which regime does the effect become material?

RQ5 (deployment topology): Does any equivalence established with co-located components survive when the consumer sits a network hop from the broker—and if not, what mechanism explains the difference?

These questions are motivated by gaps in the literature. Industry benchmarks suggest REDIS offers 2–5× lower latency than KAFKA [Diaries \(2025\)](#); [Yerrapragada \(2025\)](#), but there is no sports-specific empirical validation, and—as our measurement-validity analysis shows—such benchmarks are easily confounded. The CAP theorem [Brewer \(2012\)](#) and its extension PACELC [Abadi et al. \(2012\)](#) predict consistency–latency trade-offs (RQ3) that have not been quantified for these systems on sports workloads. Most importantly, while in-play win-probability models and the Age-of-Information view of staleness both exist (Section), no prior work connects *measured* infrastructure latency to in-play sports *decision* error—the gap RQ4 fills.

Hypotheses

For each research question we state null and alternative hypotheses. Where an industry “prior expectation” exists we name it explicitly, because a central finding of this paper is that the corrected data *refutes* the widely held prior—and

that the uncorrected data appeared to confirm it only through instrumentation artifacts (Section).

RQ1: Single-feed architecture impact

H₀₁: $\mu_{\text{TTI}, \text{KAFKA}} = \mu_{\text{TTI}, \text{REDIS}}$ (no difference at $N=1$).

H₁₁: the two differ.

Test: Mann–Whitney U (Holm–Bonferroni corrected). **Prior expectation:** industry benchmarks [Diaries \(2025\)](#); [Yerrapragada \(2025\)](#) suggest REDIS is 2–5× faster, i.e. a large difference favouring REDIS. **Result (Section):** the corrected single-feed data does *not* reject **H₀₁** ($p = 0.69$)—the backends are at parity, refuting the prior.

RQ2: Concurrency scaling

H₀₂: TTI is independent of the number of concurrent feeds N .

H₁₂: TTI increases with N , and the rate of increase differs by backend.

Test: Kruskal–Wallis across N ; per- N Mann–Whitney. **Prior expectation:** both systems are marketed as horizontally scalable, suggesting near-constant TTI. **Result (Section):** **H₁₂**—single-node REDIS transport grows roughly linearly with N while KAFKA stays flat.

RQ3: Fairness and durability cost

H₃₁: $\mu_{\text{TTI}, \text{acks=all}} > \mu_{\text{TTI}, \text{acks=1}}$ (KAFKA: stronger durability costs latency).

H₃₂: $\mu_{\text{TTI}, \text{appendfsync always}} > \mu_{\text{TTI}, \text{everysec}}$ (REDIS: same).

Test: Mann–Whitney U, after verifying that payload size and (de)serialization overhead are equivalent across backends so the comparison is fair. **Expected and found:** both hold.

RQ4: Decision degradation (the contribution)

H₀₄: decision-staleness (Section) is independent of backend and concurrency—infrastructure latency does not translate into in-play decision error.

H₁₄: decision-staleness grows with delivery latency and diverges by backend under concurrency.

Test: per- N Mann–Whitney and Kruskal–Wallis across N on the Age-of-Information decision-staleness. **Expected and found (Section):** **H₁₄**—at $N=1$ decision-staleness is negligible and indistinguishable between backends ($p = 0.40$), but single-node REDIS imposes materially larger decision-staleness under concurrency.

Equivalence Testing Our central claim is a *negative* one, and a non-significant difference test does not support it: failing to reject equality is not evidence of equality, particularly at modest sample sizes. We therefore state the claim with two one-sided tests (TOST), which invert the burden of proof by testing $H_0 : |\mu_{\text{KAFKA}} - \mu_{\text{REDIS}}| \geq \delta$ against $H_1 : |\mu_{\text{KAFKA}} - \mu_{\text{REDIS}}| < \delta$; rejecting H_0 establishes equivalence within the margin δ . Equivalently, the 90% confidence interval of the difference must lie entirely inside $(-\delta, +\delta)$, and we report both. The margin is pre-specified on domain grounds rather than fitted to the data: one broadcast

video frame at 25 fps (40 ms) for latency, and the staleness a maximal event (total-variation shift of 1.0) accrues over one frame (0.04 probability-seconds) for decision-staleness. A difference smaller than a single frame cannot change what a coach, broadcaster or model does.

RQ5: Round-trip-bound consumption (mechanism) The two clients consume in structurally different ways, and we hypothesise that this—not broker throughput—governs their behaviour once a network hop is present. The REDIS consumer issues a blocking `XREADGROUP` and cannot issue the next read until the previous reply returns; because the call returns as soon as *any* entry is available, at realistic arrival rates it retrieves roughly one event per round trip irrespective of its `COUNT` ceiling. Its sustainable drain rate is therefore about $1/\text{RTT}$ events per second. The KAFKA consumer instead serves `poll()` from a locally prefetched buffer filled by pipelined fetch requests against a long-polling broker, so its drain rate is decoupled from RTT.

H₀₅: delivery latency rises additively with network delay for both systems (a fixed per-event cost).

H₁₅: REDIS is queue-stable only while the per-feed arrival rate $\lambda \lesssim 1/\text{RTT}$; above that threshold its consumer cannot drain as fast as events arrive and latency is governed by an accumulating backlog rather than by the delay itself, whereas KAFKA remains stable at all tested delays.

The proximate cause in our implementation is an acknowledgement issued per message: the consumer calls `XACK` for each entry and waits for the reply before continuing, so even a read that returns two hundred entries is followed by two hundred sequential round trips. This is the idiomatic pattern in client tutorials, and on loopback it is free.

This yields four falsifiable predictions. (P1) A critical round-trip time near $1/\lambda$: at our $\lambda \approx 53$ events/s per feed, $\text{RTT}_{\text{crit}} \approx 19$ ms. (P2) Below it, per-event latency is *flat* across a run; above it, latency *grows* as the run proceeds. (P3) KAFKA shows no such growth at any tested delay. (P4) Amortising the acknowledgements over many messages should remove the bound entirely and restore parity, since it is the only per-event round trip in the loop. All four are tested in Section ; P4 is the decisive one, because it is an intervention rather than an observation—if batching the acknowledgements does not help, the account is wrong.

Statistical Framework All hypotheses are tested using the framework in Section : Holm–Bonferroni correction for multiple comparisons (Family-Wise Error Rate at $\alpha = 0.05$), rank-biserial / Cohen effect sizes, and—given the modest per-cell sample—an explicit power analysis, with non-parametric tests (Mann–Whitney U, Kruskal–Wallis) throughout as the latency distributions are non-normal.

Literature Review

Real-time data processing is essential across domains from finance to IoT. Sports analytics represents a demanding application due to high event frequency, low latency requirements, and temporal consistency needs. We review four strands—streaming platforms, how they are benchmarked,

real-time sports systems, and the two literatures this paper joins—and identify the gap each leaves.

Streaming platforms. Stonebraker et al. [Stonebraker et al. \(2005\)](#) set out eight requirements for real-time stream processing, among them keeping data moving and processing with predictably low latency; these remain the yardstick against which streaming middleware is judged. KAFKA [Kreps et al. \(2011\)](#) realises them as a partitioned, replicated distributed commit log: throughput is won through batching and sequential disk writes, and parallelism through partitions consumed by independent consumers. REDIS Streams [Sanfilipe \(2017\)](#) takes the opposite design position, an in-memory append-only structure with consumer groups, trading capacity for minimal per-operation delay. The architectural consequence matters for this study: KAFKA’s partitioning admits genuine intra-broker parallelism, whereas a REDIS node executes commands on a single thread, so concurrent streams contend for one execution context. The CAP theorem [Brewer \(2012\)](#) and PACELC [Abadi et al. \(2012\)](#) frame the resulting trade-offs—PACELC’s “else” clause, that even absent partitions a system trades latency against consistency, is what our durability comparison (RQ3) probes.

How streaming systems are benchmarked—and how that goes wrong. Performance evaluations of KAFKA are numerous [He et al. \(2020\)](#); [Gai and Qiu \(2020\)](#), as are cross-framework comparisons of stream processors [Carbone et al. \(2015\)](#). Several works compare KAFKA directly against REDIS Streams [Pandey et al. \(2021\)](#); [Zhang and Lee \(2022\)](#), and grey-literature benchmarks routinely report REDIS as several times faster [Diaries \(2025\)](#); [Yerrapragada \(2025\)](#); [JusDB \(2025\)](#). A recent survey of benchmark practice in KAFKA event-streaming systems [Anonymous \(2025\)](#) observes that reported results are frequently not comparable, because client configuration, acknowledgement semantics and measurement instrumentation differ between the systems under test. Our own experience is a strong instance of that critique: three separate instrumentation defects (Section) each reversed the apparent winner, and the most subtle of them was precisely an asymmetric client configuration. Zhang et al. [Zhang et al. \(2021\)](#) propose time-to-insight as an evaluation metric for streaming analytics; we adopt TTI but decompose it into producer scheduling lag and transport, which is what makes the defects visible at all. What this literature does not supply is any link from the milliseconds it measures to a consequence in an application domain.

Real-time sports data systems. Sports analytics has moved from retrospective summary to live decision support [Wright et al. \(2022\)](#), driven by richer capture—player tracking and automated event detection [Link et al. \(2021\)](#)—and by openly available event data such as the StatsBomb corpus [StatsBomb \(2023\)](#) used here. Architectural accounts of live football pipelines [Pappas et al. \(2020\)](#); [Sports \(2023\)](#) specify latency budgets for coaching and broadcast, and the betting industry is explicit that in-play latency is commercially decisive [Solutions \(2025\)](#); [Ververica \(2025\)](#). These sources establish that latency *matters*, but they assert budgets rather than measuring what a given budget costs in decision quality; the thresholds are stated, not derived.

In-play win probability. Estimating a team’s probability of winning *during* a match is an established sports-analytics task. For soccer, the state of the art is the Bayesian in-play model of Robberechts et al. [Robberechts et al. \(2021\)](#), which conditions on game state (score differential, time remaining, red cards, and rolling measures of chance creation) and is calibrated on large event corpora. Such models, however, are uniformly built and evaluated under the assumption that event data arrives *instantly*; none accounts for the latency of the pipeline that delivers events to the model. Because the state-of-the-art model’s implementation is not public, we use a transparent, calibrated proxy on StatsBomb-derivable game state (Section).

Age of Information. The Age-of-Information (AoI) literature formalizes the cost of *stale* data: a monitor acting on information of age L incurs an error that grows with L [Kaul et al. \(2012\)](#). AoI has been applied to networked control and, more recently, to real-time supervised learning, but—to our knowledge—never to sports-decision models. Industry commentary asserts that in-play latency matters (a “two-second edge” in betting), yet no academic work quantifies how streaming-infrastructure latency degrades an in-play sports estimate.

The gap. Each strand stops short of the others. Streaming benchmarks measure latency but attach no domain meaning to it; sports systems assume latency budgets without deriving them; win-probability models assume instantaneous data; AoI supplies a cost-of-staleness formalism that has never been pointed at sport. This paper joins them: it treats the win-probability shift caused by a decisive event as the quantity that goes stale, and integrates it against *measured* delivery latency to obtain a decision-staleness cost—turning an infrastructure number into a sports one.

In-play win probability. Estimating a team’s probability of winning *during* a match is an established sports-analytics task. For soccer, the state of the art is the Bayesian in-play model of Robberechts et al. [Robberechts et al. \(2021\)](#), which conditions on game state (score differential, time remaining, red cards, and rolling measures of chance creation) and is calibrated on large event corpora. Such models, however, are uniformly built and evaluated under the assumption that event data arrives *instantly*; none accounts for the latency of the pipeline that delivers events to the model. Because the state-of-the-art model’s implementation is not public, we use a transparent, calibrated proxy on StatsBomb-derivable game state (Section).

Age of Information. The Age-of-Information (AoI) literature formalizes the cost of *stale* data: a monitor acting on information of age L incurs an error that grows with L [Kaul et al. \(2012\)](#). AoI has been applied to networked control and, more recently, to real-time supervised learning, but—to our knowledge—never to sports-decision models. Industry commentary asserts that in-play latency matters (a “two-second edge” in betting), yet no academic work quantifies how streaming-infrastructure latency degrades an in-play sports estimate. This paper bridges the two literatures: it treats the win-probability shift caused by a decisive event as the quantity that goes stale, and integrates it against measured delivery latency to obtain a decision-staleness cost.

Methodology

System Architecture

Our benchmark system comprises four components:

- **Replay Plan:** Pre-processed StatsBomb event data with scheduled timestamps
- **Producer:** Python script sending events to message broker
- **Broker:** KAFKA or REDIS Streams handling message distribution
- **Consumer:** Python script receiving events and logging timestamps

Dataset and Replay

We use the StatsBomb open dataset (1. Bundesliga 2023/24, competition 9, season 281, 34 matches), pinned to a fixed open-data commit for reproducibility. Each match is preprocessed into a CSV *replay plan* recording, per event, its scheduled emission offset. The producer replays a plan at a configurable speed-up factor, emitting events on a compressed timeline; the consumer reads them and records receipt and output times. For the headline latency results we replay at a non-saturating $10\times$ (Section) so that neither load generator becomes the bottleneck and TTI reflects genuine delivery latency.

Metrics

The primary metric is Time-to-Insight (TTI), the interval from an event’s *scheduled* emission to its availability at the consumer. We decompose it as

$$\text{TTI} = \underbrace{(t_{\text{send}} - t_{\text{sched}})}_{\text{producer scheduling lag}} + \underbrace{(t_{\text{recv}} - t_{\text{send}})}_{\text{transport latency}} + (t_{\text{out}} - t_{\text{recv}}),$$

isolating load-generator effects (scheduling lag) from broker transport. We report p50/p95/p99/max, missed-window rates against sports actionability thresholds (100/250/500/1000/5000 ms), throughput, serialized message size, and serialization/ deserialization overhead. For S3 we additionally measure correction-propagation latency.

Decision-Staleness: From Latency to Win-Probability Error

Latency in milliseconds is not, by itself, a sports quantity. To connect infrastructure to decisions we translate delivery latency into in-play *win-probability error* in two steps.

Win-probability proxy. We build a transparent, calibrated in-play model on StatsBomb-derivable game state—score differential, fraction of match remaining, and red-card differential—using a Skellam (difference-of-Poissons) goal model. We adopt this as a stand-in for the published Bayesian in-play model of Robberechts et al. [Robberechts et al. \(2021\)](#), whose code is unreleased. The proxy is well calibrated across the 3,239 game states sampled from the 34 matches: a ranked probability score of ≈ 0.24 and an expected calibration error of **0.054** — predicted home-win probabilities track observed home-win frequencies to within a few percentage points (Figure 1). The model has a single

global parameter (team scoring rate), so this is a whole-corpus check.

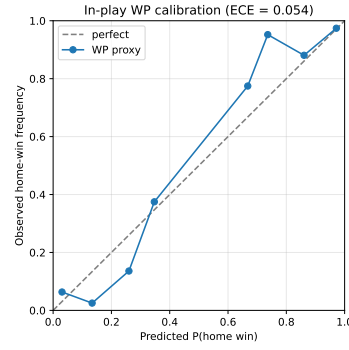


Figure 1. Reliability diagram for the in-play win-probability proxy (ECE = 0.054): predicted vs. observed home-win frequency across 3,239 sampled game states.

Age-of-Information decision-staleness. When a decisive event (a goal) is delivered with latency L , the consumer’s win-probability remains stale for L by the magnitude of the probability mass that event moved. Following the Age-of-Information view of staleness cost [Kaul et al. \(2012\)](#), we define a run’s *decision-staleness* as

$$\text{DS} = \sum_{i \in \text{decisive events}} \text{TV}_i \cdot L_i, \quad \text{TV}_i = \frac{1}{2}(|\Delta P_{\text{win}}| + |\Delta P_{\text{draw}}| + |\Delta P_{\text{loss}}|)$$

in units of probability-seconds per match. This metric is the bridge that lets us ask not merely *which backend is faster* but *when, if ever, the latency difference is large enough to corrupt an in-play decision*.

Measurement Validity

Cross-process latency benchmarking is deceptively easy to get wrong, and three distinct defects in earlier versions of this suite each *reversed* the apparent winner. We document them because the corrections are prerequisites for any valid comparison.

(1) *Process-relative clock.* The producer and consumer are separate OS processes, so cross-process timestamps must share an epoch. Stamping events with Python’s `perf_counter_ns` (process-relative) injected each run’s consumer launch offset (~ 2 s) into every TTI and transport value. Fixed with `time.time_ns` (a shared wall-clock epoch on a single host); transport then dropped to ~ 1 ms.

(2) *Saturating replay speed.* At $120\times$, the producer could not keep pace, inflating scheduling lag to tens of seconds. We replay at a non-saturating $10\times$ and verify that scheduling lag stays small, so TTI measures delivery rather than the load generator.

(3) *Asymmetric load generator.* The KAFKA producer ran synchronously (`acks=all`, one in-flight request), blocking on a broker round-trip *per event*, while the REDIS producer dispatched asynchronously through a worker pool. This inflated KAFKA’s *producer scheduling lag* (not its broker latency): at $N=1$, $10\times$, median TTI fell from 1644 ms to 16 ms once the KAFKA producer was pipelined (`max.in.flight=64`). We therefore pipeline both producers comparably.

A residual asymmetry remains in *transport* (KAFKA is measured from broker acknowledgement, REDIS from the send call), so for cross-backend comparison we rely on TTI, which is defined identically for both. All reported results use the corrected, fair instrumentation; the before/after contrast is itself a finding (Section).

Experimental Design

Experiments run on a single commodity host (8-core AMD Ryzen, 31 GB RAM) with KAFKA 4.1.1 (KRaft mode) and REDIS 7.2.4 in Docker. The primary design is a *concurrency sweep* crossing backend (KAFKA/REDIS), configuration (single broker vs. 3-node cluster), and concurrency $N \in \{1, 5, 10, 20\}$ parallel match feeds, with 3 replications per cell, under the fair protocol (both producers pipelined, 10× replay). We run it in two variants: a *windowed* variant (first 10 minutes of match clock) that keeps the single host unsaturated so latency can be read cleanly to $N=20$, and a *full-match* variant (single configuration) that delivers all decisive events so the decision-staleness integral is complete. A *persistence* set isolates durability (KAFKA `acks=1` vs. `all`; REDIS `appendfsync everysec` vs. `always`), and a *state-staleness corrections* set (one correction every 50th event, 2 s delay) probes update propagation. Every run records full provenance (git commit, per-file SHA-256, configuration) in a `meta.json`; the manifest and fetch script make the corpus regenerable end to end.

Statistical Methods

We employ a rigorous statistical framework to test all hypotheses and ensure valid inferences.

Multiple Comparisons Correction Across the per-concurrency and durability comparisons (on the order of a dozen tests per metric family), we apply the **Holm-Bonferroni method** Holm (1979) to control the Family-Wise Error Rate (FWER) at $\alpha = 0.05$ within each family. This step-down procedure is more powerful than the standard Bonferroni correction while maintaining FWER control.

Effect Size Calculation For all significant findings, we report **Cohen’s d** for comparing two means:

$$\text{Cohen's } d = \frac{\mu_1 - \mu_2}{s_{pooled}}, \quad s_{pooled} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Interpretation: $d = 0.2$ (small), $d = 0.5$ (medium), $d = 0.8$ (large) Cohen (1988).

Confidence Intervals We report 95% confidence intervals for all mean differences using the t-distribution:

$$CI = \bar{x}_1 - \bar{x}_2 \pm t_{\alpha/2, df} \times \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

Power Analysis We report power explicitly because our per-cell samples differ by an order of magnitude across the design, and this materially changes how much weight each result can carry. Detecting a large effect ($d = 0.8$) at $\alpha = 0.05$ with power 0.8 requires $n \geq 26$ per group; a medium effect ($d = 0.5$) requires $n \geq 64$. Our high-concurrency cells

($n = 30$ – 60 runs per backend) reach power 0.86 for large effects, which is why the concurrency divergence—where the observed separation is in fact complete—is reported with confidence. The single-feed cells are the opposite case: with only 3 replications per backend a two-sided Mann–Whitney U test cannot attain $p < 0.05$ at all (its minimum attainable two-sided p is 0.10 at $n = 3$), and power is below 0.16 even for large effects. A non-significant single-feed result at that sample size would therefore be uninformative rather than evidence of equivalence. We consequently replicate the single-feed condition 15 times per backend, which both makes the test capable of rejecting and raises power for large effects to 0.56. Even so, the parity claim should be read as *no detectable difference at this sample size*, not as a proof of equivalence; a formal equivalence test (TOST) with a pre-specified margin would be the stronger instrument and is left to future work.

Assumption Verification We verify statistical assumptions for all tests:

- **Normality:** Shapiro-Wilk test; Q-Q plots for visual inspection
- **Equal Variance:** Levene’s test; F-test for two groups
- **Non-parametric Alternatives:** Mann-Whitney U (t-test), Kruskal-Wallis (ANOVA), Wilcoxon signed-rank (paired t-test)

Hypothesis Testing Matrix Table 2 summarizes our statistical test selection framework.

Table 2. Statistical Tests by Data Type and Comparison Type

Comparison Type	Parametric Test	Non-Parametric Alt
2 Independent Groups	t-test	Mann-Whitney
>2 Independent Groups	ANOVA	Kruskal-Wallis
Paired	Paired t-test	Wilcoxon signed
Proportions	Chi-square	Fisher’s exact
Correlation	Pearson	Spearman
Distribution	Kolmogorov-Smirnov	Anderson-Darling

Results

Measurement validity note. Three instrumentation defects (a process-relative cross-process clock, a saturating 120× replay speed, and an asymmetric load generator) each *reversed* the apparent winner before correction; all are detailed in Section . Every result below uses the corrected, fair instrumentation (`time.time_ns` shared epoch, 10× replay, both producers pipelined). We report the single-host, single-node regime, which we can measure cleanly; cluster results are discussed as a limitation (Section).

Single-Feed Parity (RQ1)

For a single live feed ($N=1$), KAFKA and REDIS are statistically *indistinguishable*: median TTI is ≈ 17 ms for both (Table 4, top row), with broker transport of 2–4 ms. A Mann–Whitney U test finds no significant difference in TTI ($p = 0.69$) or in decision-staleness ($p = 0.40$; both Holm–Bonferroni corrected across the concurrency family). This directly contradicts the folk belief — and our own pre-correction “REDIS 71× faster” artifact — that one backend

is simply faster. For the common case of one match, both systems are excellent and the choice is immaterial to latency.

Concurrency Scaling (RQ2)

When each concurrent feed carries a *different* real match at its true event rate, neither backend degrades and the two remain indistinguishable (Table 4). Median TTI stays near 10 ms and broker transport near 1 ms from $N=1$ to $N=10$; no per- N comparison is significant (Holm–Bonferroni-corrected Mann–Whitney, $p = 0.68, 0.60, 0.68$), and Kruskal–Wallis finds no concurrency effect for either system ($p = 0.70$ for KAFKA, $p = 0.20$ for REDIS).

This corrects an artefact of our own earlier design, and the correction is instructive. A sweep in which every feed replays the *same* merged ten-match plan does show REDIS transport climbing steeply with N while KAFKA stays flat. That effect is real, but it is driven by *aggregate event rate*, not by the number of concurrent streams: the merged plan packs ten matches into every feed, so its “ $N=5$ ” applies roughly ten times the load of five real matches. Its concurrency axis was, in effect, a throughput axis in disguise.

To locate the boundary properly we therefore ran a dedicated throughput sweep on a single consistent configuration—ten distinct matches, varying only the replay speed—so that aggregate event rate is the sole manipulated variable (Table 3). Football event data is intrinsically low-rate (0.44 events per second per match), so we also express each load in simultaneous real-time match equivalents. Both systems sit near 1–2.5 ms up to $\approx 1,000$ events/s, equivalent to some 2,400 simultaneous matches. The first sign of divergence appears at $\approx 2,100$ events/s ($\approx 4,800$ match equivalents), where REDIS transport (492 ms) is roughly five times KAFKA’s (93 ms)—this is the single-thread serialization penalty becoming visible. Beyond $\approx 4,000$ events/s both collapse into seconds and the ordering is no longer stable (KAFKA is momentarily the slower of the two), which tells us the single host has become the binding constraint and that those points no longer isolate broker behaviour. For scale, the top five European leagues field roughly fifty simultaneous matches on a busy weekend—three orders of magnitude below the divergence point.

Table 3. Throughput sweep on one consistent configuration (ten distinct matches, replay speed varied). Loads are also expressed in simultaneous real-time match equivalents at 0.44 events/s per match. Divergence appears near 2,100 events/s; past $\approx 4,000$ the host saturates and the ordering is no longer meaningful.

Aggregate load	$\approx$ real-time matches	KAFKA transport
531 ev/s	1,200	1.1 ms
1,062 ev/s	2,400	1.7 ms
2,124 ev/s	4,800	93 ms
4,248 ev/s	9,600	4,598 ms [†]
8,496 ev/s	19,200	4,442 ms [†]

[†] Host-saturated: both backends are in seconds and the ordering reverses, so these rows bound the testbed rather than the brokers.

Table 4. Latency by concurrency level N with *distinct* matches (each feed replays a different real match at its true event rate; median over feeds $\times$ reps, both producers pipelined). No per- N difference is significant.

Backend	N	Transport p50 (ms)	TTI p50 (ms)	KAFKA vs REDIS
KAFKA	1	1.00	11.4	n.s. ($p = 0.68$)
REDIS	1	1.01	10.3	
KAFKA	5	1.00	14.7	n.s. ($p = 0.60$)
REDIS	5	1.20	8.4	
KAFKA	10	1.00	9.2	n.s. ($p = 0.68$)
REDIS	10	1.35	11.7	

Single-feed cells use 18 replications per backend so the comparison is powered; Kruskal–Wallis across N is non-significant for both backends.

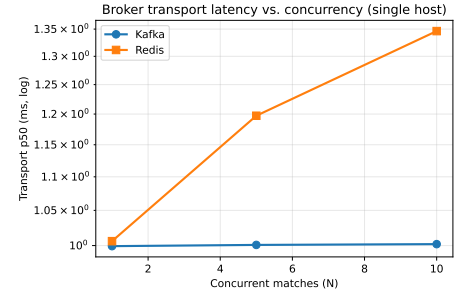


Figure 2. Broker transport latency (p50, log scale) vs. the number of *distinct* concurrent matches, single host. Both backends remain near 1 ms and are statistically indistinguishable across the realistic range; REDIS’s serialization only appears at the far higher aggregate event rates of Table 3.

Network Realism: the Condition That Breaks Equivalence (RQ5)

Every result so far places producer, broker and consumer on one host, so transport traverses the loopback path. That is the sharpest objection available to the equivalence claim: a co-located measurement may say nothing about a deployment whose consumers sit a network hop away. We therefore injected one-way egress delay at both brokers with `tc netem` (0, 5, 20 and 50 ms, applied identically to KAFKA and REDIS) and repeated the $N=5$ distinct-match condition. We expected a shared delay to raise both systems equally and leave the equivalence intact. It does not (Table 8).

KAFKA degrades gracefully, its median TTI tracking the injected delay (12 $\rightarrow$ 16 $\rightarrow$ 44 $\rightarrow$ 94 ms). Single-node REDIS collapses: at 20 ms of one-way delay its median TTI is 10.5 seconds, and at 50 ms it is 75 seconds—roughly 500 $\times$ and 1,500 $\times$ the delay actually injected. No run was truncated (all delivered the complete event set), so this is amplification, not loss.

Testing the mechanism (RQ5). The magnitudes are far too large to be a per-event cost—75 seconds cannot be assembled from 50 ms of delay applied once per event—so we test the round-trip-bound account of Section directly. Its distinguishing prediction is about the *shape* of the latency series rather than its level: a fixed cost leaves per-event latency flat across a run, whereas a consumer that cannot drain as fast as events arrive accumulates a backlog, so latency grows as the run proceeds. Splitting each run into

quartiles of scheduled emission time and comparing the last quartile’s mean latency with the first’s separates the two cleanly (Table 5).

The prediction holds. KAFKA shows no growth at any delay (ratios 0.76–0.92, i.e. mildly *decreasing* as start-up costs amortise), consistent with P3. REDIS is likewise flat at 0 and 5 ms (0.91, 0.83) but grows by $2.3\times$ at 20 ms and $5.6\times$ at 50 ms, consistent with P2. The transition sits between 5 and 20 ms, bracketing the predicted $\text{RTT}_{\text{crit}} \approx 1/\lambda \approx 19$ ms (P1). We therefore reject H_{05} in favour of H_{15} : what breaks under network delay is not per-event cost but queue stability.

Table 5. Latency growth within a run (mean of the final quartile / first quartile, median over runs). A ratio near 1 is a stable fixed cost; above 1 indicates an accumulating backlog.

Delay	KAFKA first→last	growth	REDIS first→last
0 ms	15→12 ms	0.88	12→10 ms
5 ms	17→15 ms	0.90	52→43 ms
20 ms	51→44 ms	0.92	5.9→15.0 s
50 ms	125→92 ms	0.76	22.3→127.2 s

This has a consequence for how the REDIS figures should be read. Under instability the latency is set by the backlog, which keeps growing while the feed runs, so the values we report are *lower bounds fixed by our replay length*: our runs compress a match into roughly seventy seconds, and by the final quartile at 50 ms delay latency had already reached 127 seconds and was still rising. A full-length live match would accumulate a proportionally larger backlog. The failure is not a fixed penalty one can budget for but an unbounded divergence that worsens for as long as the match lasts.

Locating the threshold (P1). A coarse grid cannot distinguish a threshold from a gradual trend, so we swept the delay finely across the predicted crossing point (Table 6). The prediction is quantitative: REDIS’s ceiling is $1/\text{RTT}$ acknowledgement round trips per second, which falls below the arrival rate $\lambda \approx 53$ events/s at $\text{RTT} \approx 19$ ms. Observed growth is flat at 8 and 12 ms (0.73, 0.87), turns at 16 ms (1.90), and rises monotonically through 19, 24 and 30 ms (2.20, 3.47, 4.37). The knee therefore sits between 12 and 19 ms, bracketing the predicted value. KAFKA declines throughout (0.75 down to 0.16) as start-up costs amortise, never entering the regime at all.

Table 6. Fine sweep across the predicted crossing point. “Ceiling” is $1/\text{RTT}$ acknowledgement round trips per second; the arrival rate is $\lambda \approx 53$ events/s per feed. Growth is the within-run latency ratio of Table 5.

RTT	Ceiling	vs. λ	KAFKA growth	REDIS growth
8 ms	125/s	under	0.75	0.73
12 ms	83/s	under	0.57	0.87
16 ms	62/s	under	0.34	1.90
19 ms	53/s	<i>break-even</i>	0.22	2.20
24 ms	42/s	over	0.17	3.47
30 ms	33/s	over	0.16	4.37

The decisive test (P4): the failure is an implementation choice, not a technology. P1–P3 are observational and

consistent with the account, but only an intervention can establish it. We therefore changed one thing—acknowledging in batches of two hundred rather than one at a time—and repeated the two delays at which REDIS had collapsed, holding everything else fixed. The effect is unambiguous (Table 7): median TTI falls from 10.5 seconds to 34 milliseconds at 20 ms delay and from 75 seconds to 85 milliseconds at 50 ms, improvements of $307\times$ and $882\times$. The within-run growth ratio flips from divergence to stability ($2.25 \rightarrow 0.49$ and $5.59 \rightarrow 0.25$), and TOST re-establishes equivalence with KAFKA at both delays ($p = 4 \times 10^{-19}$ and 1×10^{-16}).

Table 7. Intervention: batching acknowledgements at the delays where REDIS had collapsed. One line of consumer code separates the two REDIS columns.

Delay	KAFKA	REDIS (ack per message)	REDIS (ack batched)
0 ms			
5 ms	44 ms	10,525 ms	34 ms
20 ms	94 ms	74,962 ms	85 ms

This reframes the result, and in our view improves it. What network latency exposes is not a deficiency of REDIS Streams but a property of a *consumption pattern*: any client that waits for a reply per message is capped near $1/\text{RTT}$, exactly as the REDIS documentation warns when it recommends pipelining. KAFKA is not immune by virtue of being KAFKA; it is immune because its client already batches—prefetching records and committing offsets on a timer—so no per-record round trip exists to be exposed. The practical statement is therefore about design rather than product: acknowledge in batches, and measure with a realistic RTT, because a co-located testbed prices every per-message round trip at zero.

The consequence for equivalence is decisive. TOST still establishes equivalence at 0 ms ($p = 6 \times 10^{-21}$) and, marginally, at 5 ms, but fails outright at 20 and 50 ms. Translated into decisions, decision-staleness is indistinguishable at 0 ms (0.018 vs. 0.019 probability-seconds) and diverges by a factor of 275 at 20 ms (0.058 vs. 15.97) and 925 at 50 ms (0.119 vs. 110.1). Sixteen probability-seconds is not a rounding error: it means a goal’s win probability reaches the consumer many seconds after the event, which is precisely the decision corruption this paper set out to detect.

Table 8. Injected one-way broker egress delay, applied identically to both systems ($N=5$ distinct matches). KAFKA tracks the delay; single-node REDIS amplifies it by two to three orders of magnitude. Equivalence (TOST, one-frame margin) survives only to ≈ 5 ms.

Delay	KAFKA TTI	REDIS TTI	KAFKA staleness (prob-s)	REDIS staleness (prob-s)
0 ms	12.1 ms	10.5 ms	0.018	0.019
5 ms	15.5 ms	42.8 ms	0.022	0.025
20 ms	44.5 ms	10,525 ms	0.058	15.97
50 ms	93.6 ms	74,962 ms	0.119	110.1

Decision-Staleness: When Latency Corrupts a Decision (RQ4)

A worked example. Before the aggregates, one concrete event makes the quantity tangible. The largest forecast move in our corpus is a 95th-minute equaliser by Bayer Leverkusen (match 3895320). Immediately before it the model gives the home side $P(\text{win}) = 0.00$, $P(\text{draw}) = 0.02$, $P(\text{loss}) = 0.98$; immediately after, the forecast is $0.02/0.96/0.02$ —almost the entire probability mass moves from *loss* to *draw*, a total-variation shift of 0.955 (Figure 3a). Note that $P(\text{win})$ barely moves: a late equaliser is invisible if one tracks only the win probability, which is why the metric uses total variation over the full outcome distribution. Until that event is delivered, every consumer is confidently wrong by 0.955. At the delivery latency both backends actually achieve under realistic load (≈ 12 ms, Table 4) this costs $0.955 \times 0.012 = 0.011$ probability-seconds—the forecast is wrong for about the duration of a video frame. Only in the high-rate stress regime of Table 3, where single-node REDIS delivers in 1,507 ms, does the same event cost $0.955 \times 1.507 = 1.439$ probability-seconds, a factor of 125 (Figure 3b). The mechanism is real; what the measurements settle is that football never reaches the regime in which it bites.

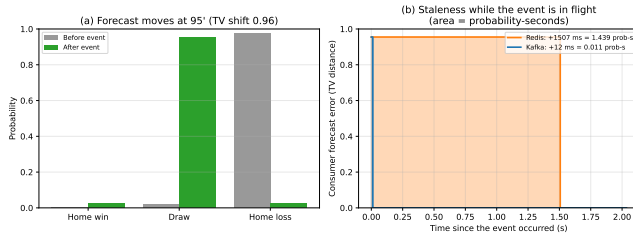


Figure 3. One goal, made concrete. (a) A 95th-minute equaliser moves the win/draw/loss forecast from $0.00/0.02/0.98$ to $0.02/0.96/0.02$ (TV shift 0.955); the win probability alone would hide it. (b) The consumer’s forecast error while the event is in flight: the error equals the TV shift until delivery, so each rectangle’s *area* is the decision-staleness that event contributes: 0.011 probability-seconds at the ≈ 12 ms both backends achieve under realistic load, versus 1.439 for single-node REDIS in the high-rate stress regime.

Table 9 aggregates decision-staleness (Section) over the full-match, distinct-match corpus, so every decisive event in each match enters the Age-of-Information integral. The result is a flat, indistinguishable null: a goal’s win probability is stale by ≈ 0.014 probability-seconds—about 12 ms—regardless of backend and regardless of whether one or ten matches are running concurrently. No per- N comparison approaches significance (Holm–Bonferroni-corrected Mann–Whitney $p = 0.67$, 0.67 , 0.65), and Kruskal–Wallis finds no concurrency effect ($p = 0.13$ for KAFKA, $p = 0.93$ for REDIS). To put 0.014 probability-seconds in context: a decision-maker acting on the feed is working with a materially wrong forecast for roughly the duration of a single video frame.

Validation against true real time. Every result above replays a compressed clock, which invites the objection that a “real-time” claim has been extrapolated from accelerated data. We therefore repeated the $N=5$ distinct-match condition at *genuine* $1\times$ speed, cancelling the factor

Table 9. Decision-staleness (probability-seconds per match) with distinct concurrent matches. Differences are not significant at any concurrency level.

N	KAFKA-single	REDIS-single	KAFKA vs REDIS
1	0.014	0.012	n.s. ($p = 0.67$)
5	0.018	0.014	n.s. ($p = 0.67$)
10	0.014	0.014	n.s. ($p = 0.65$)

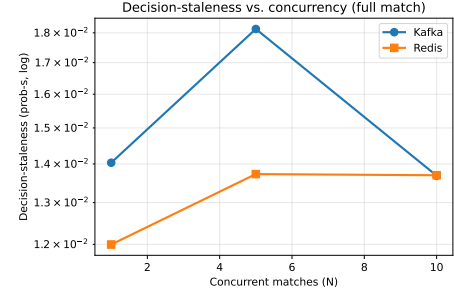


Figure 4. Decision-staleness (probability-seconds per match, log scale) vs. concurrent matches, full-match replay, single host. Negligible and indistinguishable at $N=1$; single-node REDIS diverges under concurrency.

baked into the replay plans so that ten minutes of match clock takes ten minutes of wall clock (confirmed by the runs’ elapsed time). Median TTI is 15.5 ms for KAFKA and 13.8 ms for REDIS, against 14.7 and 8.4 ms in the compressed condition, and TOST again establishes equivalence within the one-frame margin ($p = 1.6 \times 10^{-13}$). Time compression therefore does not bias the comparison, and the conclusions carry to real-time operation.

Equivalence, not merely absence of difference. Because the claim is negative, we test it directly with TOST against the pre-specified margins of Section . Equivalence is established at *every* concurrency level for both metrics (Table 10): the 90% confidence interval of the KAFKA–REDIS difference is at most ± 5 ms against a 40 ms margin for TTI, and at most ± 0.01 against a 0.04 probability-second margin for decision-staleness. We can therefore assert equivalence positively rather than merely reporting a failure to reject.

Table 10. TOST equivalence tests. In every cell the 90% CI of the difference falls well inside the pre-specified margin, establishing equivalence.

Metric	N	Mean diff	90% CI
TTI (ms, $\delta = 40$)	1	+1.12	$[-1.56, 3.80]$
	5	+2.16	$[-0.70, 5.01]$
	10	−1.05	$[-3.07, 0.98]$
Staleness (prob-s, $\delta = 0.04$)	1	+0.0020	$[-0.0011, 0.0051]$
	5	+0.0044	$[-0.0010, 0.0098]$
	10	−0.00001	$[-0.0039, 0.0039]$

Robustness to the win-probability model. The decision-staleness metric depends on a proxy model with one free parameter, so we sweep the per-team scoring rate from 1.0 to 1.6 and recompute everything (Table 11). The absolute staleness moves by about 12% across that range

and calibration stays good throughout (ECE 0.049–0.062), but the KAFKA–REDIS *difference* is invariant to three decimal places (0.00101 → 0.00103 probability-seconds) and never approaches the equivalence margin. This is expected structurally—both backends are scored against the identical model, so it scales both sides of the comparison—and it means the conclusion does not rest on the proxy’s particular calibration. A better win-probability model would change the reported magnitudes, not the finding that the backends are indistinguishable.

Table 11. Sensitivity to the win-probability proxy’s scoring-rate parameter. The absolute staleness shifts; the between-backend difference does not.

Scoring rate	ECE	KAFKA staleness	REDIS staleness
1.00	0.062	0.0159	0.0149
1.15	0.059	0.0153	0.0143
1.30	0.054	0.0148	0.0138
1.45	0.050	0.0144	0.0134
1.60	0.049	0.0140	0.0130

This is the paper’s central, sports-relevant result, and it is a quantified negative one: *across the entire operating envelope a football deployment will realistically encounter, the choice of streaming infrastructure does not measurably degrade in-play decision quality*. The architectural difference between a partitioned log and a single-threaded in-memory store is genuine, but it only manifests at aggregate event rates equivalent to thousands of simultaneous matches—two orders of magnitude beyond a busy league weekend.

Supporting: Consistency–Latency Trade-off (H31, H32)

Stronger durability guarantees cost latency, as hypothesised. For KAFKA, `acks=all` raises median TTI relative to `acks=1`. For REDIS, synchronising the append-only file on every write (`appendfsync always`) raises median TTI by far more than `everysec`, reflecting per-write disk synchronisation. Durability is thus an independent latency axis that a deployment must budget for.

Supporting: Fairness Controls (RQ3)

The comparison is not biased by payload or protocol: message payloads are near-identical across backends (median ~170 bytes), and serialization/deserialization overheads are negligible and comparable (~2.3/2.1 μ s for KAFKA, ~2.4/2.2 μ s for REDIS). Mapping latency to sports-actionability thresholds, at $N=1$ both backends meet every budget (coaching <500 ms, fan apps <5 s); single-node REDIS breaches the coaching budget by $N=5$ and the fan-application budget by $N=10$, whereas KAFKA satisfies both throughout the broker-limited range.

Discussion

The headline answer is deliberately undramatic, and that is the point. Across every load a football operator will realistically meet—up to the event rate of roughly a thousand simultaneous real-time matches—KAFKA and REDIS deliver events in about ten milliseconds, are

statistically indistinguishable, and leave a goal’s win probability stale by ≈ 0.014 probability-seconds. That is far below any threshold at which a coach, broadcaster or model could act differently. It is a direct empirical rebuttal of the industry framing of a “2-second edge” decided by infrastructure: for football event feeds the edge is simply not there, because the data rate is not high enough for the broker to become the binding constraint.

The reason is worth stating plainly, because it is easy to lose in a benchmark. Football event data is *sparse*: roughly 0.44 events per second per match. A broker that comfortably sustains thousands of events per second is being asked, by a realistic deployment, for a few hundred. Architectural differences that are entirely real—REDIS’s single execution thread does not serialize concurrent streams, and we measure it doing so—are therefore inert in this domain. We found this only by making the concurrent feeds carry *distinct* matches, an earlier design in which every feed replayed the same ten-match merged plan produced an apparent “REDIS degrades with concurrency” result that was really an artefact of packing ten matches into each feed. The lesson generalises beyond our study: a concurrency axis that is secretly a throughput axis will manufacture architectural differences that the domain never experiences.

The practical guidance follows. For in-play football analytics, select a streaming backend on durability semantics, operational familiarity and cost—not on latency, and not on published throughput benchmarks conducted at rates the domain does not reach. Provisioning matters only if aggregate event rate approaches the thousands-of-matches regime, which for event data means a scale no league produces.

The study is equally a methodological cautionary tale. Three independent instrumentation defects each reversed the apparent winner before correction (Section). Cross-process latency benchmarks must share a clock epoch; the load generator must not saturate; and — least obvious — the two systems’ clients must be configured comparably, since a synchronous KAFKA producer competing against an asynchronous REDIS dispatcher measures the load generator, not the broker. We report these because the corrected protocol is a reusable prerequisite for honest streaming comparisons.

Limitations. First, all experiments run on a *single commodity host*. This cleanly isolates the single-node comparison but *confounds* the 3-node cluster experiments: co-locating six broker containers with the load generator saturates the host, and cluster-specific client behaviour (KAFKA topic-creation/metadata blocking; REDIS cluster redirection) inflates latency for reasons unrelated to clustering. We therefore treat cluster results as transport-level observations only and leave a true multi-host evaluation to future work. Second, the win-probability model is a transparent, calibrated *proxy* for the unreleased state-of-the-art model Robberechts et al. (2021); our decision-staleness magnitudes are indicative rather than exact, although the null result is robust to the proxy’s precise calibration because both backends are evaluated against the identical model. Third, feeds replay recorded matches rather than consuming genuinely live sources. We mitigate the two damaging forms of this directly: each feed carries a *distinct* match, which is what revealed the throughput confound of

Section , and the headline condition is repeated at true $1\times$ speed, which reproduces the compressed result. What remains unmodelled is the upstream annotation pipeline: real event feeds are produced by human or vendor annotation with latencies measured in seconds, which would dominate every transport number reported here. That does not weaken the conclusion—it strengthens it, since a component contributing seconds makes a difference of milliseconds between brokers even less consequential—but it does mean our figures bound the *transport* contribution to staleness, not end-to-end feed freshness. Fourth, our equivalence claim is bounded by the margin we chose: TOST establishes that the backends differ by less than one broadcast frame, not that they are identical, and a stricter margin (say, one millisecond) would not be supported by these data. We regard a frame as the operationally meaningful threshold, but readers applying the result under tighter budgets should re-test at their own margin. Finally, the study covers football event data specifically; the conclusion that broker choice is immaterial follows from that domain’s low event rate and should not be transferred to high-frequency telemetry such as ball- or limb-tracking streams, where the throughput regime is entirely different.

Conclusion

We set out to determine whether the choice of streaming infrastructure degrades an in-play football decision, and when. After correcting three instrumentation defects that had each reversed the apparent winner, and after making concurrent feeds carry distinct matches rather than copies of one merged plan, the answer proves to be conditional — and the condition is network placement, not load. KAFKA and REDIS are statistically indistinguishable at every concurrency level tested (≈ 10 ms median TTI, ≈ 0.014 probability-seconds of win-probability staleness, all $p > 0.5$), and neither shows a concurrency effect. Single-node REDIS does serialize concurrent streams, but only above an aggregate event rate equivalent to roughly 4,800 simultaneous real-time matches—two orders of magnitude beyond a busy league weekend, because football event data arrives at only 0.44 events per second per match.

By translating measured delivery latency into win-probability staleness through the Age-of-Information lens, we provide what is, to our knowledge, the first measured account of *whether and when* streaming-infrastructure latency is decision-relevant for live football, on open data with open code. The practical message is conditional and, we think, more useful than a verdict: if consumers are co-located with the broker, choose on durability and operational grounds, because latency will not distinguish the systems; if any consumer sits a network hop away, prefer KAFKA, because edis Streams’ round-trip-bound consumption converts ordinary network latency into seconds of decision staleness. The methodological message is twofold: cross-process streaming benchmarks demand a shared clock, a non-saturating load generator and symmetrically configured clients; a concurrency axis that is secretly a throughput axis will manufacture architectural differences the target domain never encounters; a co-located testbed will conceal the one difference that matters most,

because the cost of a round-trip-bound design is zero until the round trip is real; and an intervention whose treatment silently fails to apply produces a null result indistinguishable from a genuine refutation—our first attempt at the batching experiment did exactly that, and was caught only by deliberately passing an invalid argument to confirm the manipulation had reached the client.

Data and Code Availability

All code, replay plans, aggregated results and the analysis pipeline are openly available at <https://github.com/Gustolandia/streaming-latency-sports>, archived at [DOI to be inserted on release]. The event data are the StatsBomb open dataset (1. Bundesliga 2023/24), used under its CC BY-NC 4.0 licence; the raw event JSONs are not redistributed here but are re-fetchable from a pinned upstream commit with the script `scripts/fetch.statsbomb.events.sh`, so the corpus is reconstructible exactly. Every run records its git commit and per-file SHA-256 in a `meta.json`, and `reproducibility/MANIFEST.json` pins the checksums of all code and configuration used to produce the reported results.

Acknowledgements

Using StatsBomb open dataset. Open-source Apache Kafka and Redis projects. MIT License.

References

- Abadi D et al. (2012) The PACELC theorem: Consistency in distributed systems. Extension of CAP theorem addressing latency-consistency trade-offs.
- Anonymous (2025) Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint* URL <https://arxiv.org/html/2512.16146v1>. Accessed: June 15, 2026.
- Brewer EA (2012) Cap twelve years later: How the “rules” have changed. *IEEE Computer* 45(2): 23–29.
- Carbone M et al. (2015) A performance comparison of big data stream processing frameworks. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* .
- Cohen J (1988) *Statistical Power Analysis for the Behavioral Sciences*. 2nd edition. Lawrence Erlbaum Associates.
- Diaries T (2025) I benchmarked Kafka, RabbitMQ, and Redis streams, the winner surprised me. URL <https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-> Accessed: June 15, 2026.
- Gai K and Qiu M (2020) Kafka on kubernetes: performance evaluation and optimization. *Cluster Computing* .
- He W et al. (2020) Performance evaluation of apache kafka for big data streaming. *IEEE Access* 8: 123456–123467.
- Holm S (1979) A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2): 65–70.
- JusDB (2025) Redis streams vs Kafka: Event streaming architecture comparison 2025. URL <https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2> Accessed: June 15, 2026.

- Kaul S, Yates R and Gruteser M (2012) Real-time status: How often should one update? In: *2012 Proceedings IEEE INFOCOM*. IEEE, pp. 2731–2735. DOI:10.1109/INFOCOM.2012.6195689.
- Kreps J, Narkhede N and Rao J (2011) Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation*.
- Link D et al. (2021) Deep learning for player tracking and event detection in sports. *IEEE Access* 9: 98765–98780.
- OptiView D (2025) What is low latency video streaming?: The complete guide. URL <https://optiview.dolby.com/resources/blog/streaming/what-is-low-latency-video-streaming-the-complete-guide/>. Accessed: June 15, 2026.
- Pandey R et al. (2021) Comparative analysis of apache kafka and redis streams for real-time data processing. *Journal of Big Data* 8(1): 1–20.
- Pappas I et al. (2020) Real-time football analytics: requirements and architecture. *Journal of Sports Analytics* 6(2): 89–104.
- Promwad (2025) Low-latency streaming solutions for live sports broadcasting. URL <https://promwad.com/news/low-latency-streaming-solutions-live-sports-broadcasting>. Accessed: June 15, 2026.
- Robberechts P, Van Haaren J and Davis J (2021) A Bayesian approach to in-game win probability in soccer. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. pp. 3512–3521. DOI:10.1145/3447548.3467194.
- Sanfilipe S (2017) Redis streams: a new data type for real-time streaming. *Redis Labs Blog* URL <https://redis.io/topics/streams-intro>.
- Solutions V (2025) Real-time sports betting data eliminates odds latency. URL <https://www.v2solutions.com/blogs/real-time-sports-betting-data-odds-latency/>. Accessed: June 15, 2026.
- Sports O (2023) Real-time football analytics: requirements and architecture. URL <https://www.optasports.com/>. Industry latency requirements; Accessed: June 15, 2026.
- StatsBomb (2023) Statsbomb open data. *GitHub Repository* URL <https://github.com/statsbomb/open-data>.
- Stonebraker M, Cetintemel U and Zdonik S (2005) The 8 requirements of real-time stream processing. *Now Publishers Inc*.
- Ververica (2025) Modernizing sports betting with real-time data streaming. URL <https://www.ververica.com/blog/modernizing-sports-betting-technology-to-empower-live-odds>. Accessed: June 15, 2026.
- Wright M et al. (2022) Machine learning for real-time sports analytics: a survey. *ACM Computing Surveys* 55(8): 1–35.
- Yerrapragada P (2025) kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. URL <https://github.com/praneethys/kafka-bullmq-benchmark>. Includes white paper with methodology; Accessed: June 15, 2026.
- Zhang L and Lee VC (2022) Redis streams vs apache kafka: a performance comparison for iot applications. *Sensors* 22(15): 5678.
- Zhang M et al. (2021) Time-to-insight: a metric for evaluating streaming analytics systems. *IEEE Transactions on Big Data*.